Stage 7 — Full-stack app: FastAPI + Streamlit + PostgreSQL

Resume bullet: "Built a full-stack app with FastAPI and Streamlit: REST APIs, asynchronous audio processing, PostgreSQL storage and interactive clinical documentation."

Stages 1–6 are command-line scripts. Stage 7 wraps them in a web application: a clinician uploads a recording (or pastes a transcript), the server runs the whole pipeline in the background, results are stored in PostgreSQL, and the clinician reviews, edits and approves the SOAP note in a browser.

1. Architecture

```
Browser
  |  (Streamlit renders the page)
streamlit_app.py  ----HTTP/JSON (httpx)---->  FastAPI  app/main.py      (uvicorn, one event loop)
                                                |  validate (Pydantic), save file to uploads/
                                                |  INSERT row  ---------------------->  PostgreSQL
                                                |  return 202 Accepted + id              consultations
                                                v                                        note_versions
                                       BackgroundTasks -> run_pipeline()  (app/pipeline.py, async)
                                                |  for each stage: asyncio.create_subprocess_exec(
                                                |      python transcribe.py / diarize.py / ... )
                                                |  UPDATE status/current_stage after every stage
                                                v
                                       outputs/consult_<id>_*.json  -> results copied into JSONB columns
```

Three layers, each with one job: **Streamlit** = presentation only (it never touches the database), **FastAPI** = business rules, validation and job orchestration, **PostgreSQL** = durable state. Because the UI only uses the REST API, the same API could serve a React app, a mobile app or a hospital system.

2. FastAPI — theory

ASGI, uvicorn and the event loop

FastAPI is an **ASGI** framework (Asynchronous Server Gateway Interface — the async successor of WSGI). **uvicorn** is the ASGI server: it owns the network socket and an **asyncio event loop**. The loop runs many tasks on one thread by switching between them whenever one of them *awaits* something slow (network, disk, a child process). This is **cooperative concurrency**: a task must *await* to give others a turn; code that blocks without awaiting freezes every request.

async def vs def endpoints

Endpoint style	Where FastAPI runs it	Use when
async def	directly on the event loop	all slow work is awaited (async DB driver, httpx.AsyncClient)
def	in a thread pool (via anyio), loop stays free	you call blocking libraries, e.g. sync SQLAlchemy

Our CRUD endpoints are plain **def** because they use synchronous SQLAlchemy; FastAPI runs them in its thread pool, so they never block the loop. The long work (the pipeline) is **async**.

Pydantic models

Request bodies (`TranscriptIn, NoteIn, ApproveIn`) and responses (`ConsultationSummary/Detail, NoteVersionOut`) are Pydantic classes. FastAPI uses them to parse JSON, validate it (min/max length, types) and return **422** automatically on bad input, to filter the response (only declared fields are sent) and to generate the OpenAPI schema shown at ***/docs***. `from_attributes=True` lets Pydantic read fields straight from an ORM object.

Dependency injection

`db: Session = Depends(get_db)` — FastAPI calls the generator `get_db`, injects the session, and runs its `finally` block after the response: one session per request, always closed, and easy to replace with a test database.

HTTP status codes we use (REST semantics)

Code	Meaning	Where
202 Accepted	request accepted, work continues in background	POST /consultations/audio, /transcript
201 Created	a new resource was created	POST /consultations/{id}/notes
204 No Content	success, nothing to return	DELETE /consultations/{id}
404 Not Found	id doesn't exist	any /{id} route
409 Conflict	valid request, wrong state	editing before completion / after approval
413 / 415	file too large / unsupported type	audio upload
422 Unprocessable	validation failed	Pydantic, malformed transcript

Resource-oriented URLs: nouns (`/consultations/{id}/notes`), HTTP verbs for actions (GET reads and is safe, POST creates, DELETE removes). `/approve` is a deliberate action endpoint because approval is a state transition, not a field anyone may set.

Lifespan

The `lifespan` context manager runs once at start-up: create tables, mark jobs that a previous server left 'running' as failed, create `uploads/`.

3. Asynchronous audio processing

The 202 + polling pattern

The pipeline takes ~1–3 minutes (Whisper, pyannote, BioBERT, then Qwen3-4B). An HTTP request must not wait that long (timeouts, frozen UI). So the upload endpoint saves the file, inserts a row with status **queued**, schedules the job and immediately returns **202** with the id. The client polls `GET /consultations/{id}` to see `status, current_stage, stages_done/stages_total`. States: queued → running → completed | failed.

How the job runs

```
async with _one_at_a_time:                          # asyncio.Semaphore(1)
    for name, cmd in stages:
        await asyncio.to_thread(_update, id, status="running", current_stage=name, stages_done=i)
        proc = await asyncio.create_subprocess_exec(sys.executable, *cmd, cwd=ROOT, stdout=PIPE, stderr=STDOUT
        output, _ = await proc.communicate()       # loop is free while the child works
        if proc.returncode != 0: raise RuntimeError(...)
    await asyncio.to_thread(_save_results, ...)
```

- **BackgroundTasks**: FastAPI runs the task after sending the response. Because `run_pipeline` is *async*, it runs on the event loop as a coroutine.
- **asyncio.create_subprocess_exec**: each stage is the existing script started as a child process. `await proc.communicate()` suspends our coroutine (not the server) until the child exits. The heavy CPU/GPU work happens in another OS process, so the API stays responsive — we measured a 12 ms response for GET /consultations while Whisper was running.

- **Why processes, not imports?** The scripts stay unchanged and runnable by hand; each stage's model memory (Qwen alone ~8 GB) is returned to the OS when its process exits; a crash in a stage can't kill the server; exit codes give a clean success/failure signal. Cost: models reload every job (a few seconds each).
- **asyncio.Semaphore(1)**: only one pipeline at a time — two Qwen copies would not fit in laptop memory. Other jobs wait at `async with` with status 'queued' (a simple in-process queue).
- **asyncio.to_thread**: our DB helpers are synchronous; running them in a worker thread keeps the event loop unblocked even for those few milliseconds.
- **Error handling**: any failure stores the last 1,500 characters of the stage's output in `error`; the full output of every stage is in `logs/consult_<id>.log`.
- **Restart safety**: in-process jobs die with the server, so on start-up `mark_interrupted_jobs()` marks leftover queued/running rows as failed instead of leaving them 'running' forever.

Concurrency vs parallelism (classic interview topic)

Concurrency = making progress on many tasks by interleaving them (one event loop handling many requests). **Parallelism** = literally running at the same time on several cores. asyncio gives concurrency for I/O-bound work; CPU-bound work (model inference) needs processes or threads that release the GIL. We combine both: asyncio for the web server, separate OS processes for inference.

4. PostgreSQL storage

Schema

```
consultations                                        note_versions
-------------                                        -------------
id PK, title, source_type, input_path                id PK
status, current_stage, stages_done, stages_total     consultation_id FK -> consultations.id ON DELETE CASCADE
error                                                version (1 = LM, 2+ = clinician)
transcript JSONB, entities JSONB, soap_facts JSONB   content, author, comment, created_at
generated_note TEXT, grounding JSONB, generation JSONB
review_status, approved_by, approved_at
created_at, updated_at  (timestamptz)
```

- **Relational where it matters, JSONB where data is nested**: status, review state and note versions are real columns/rows (filterable, constrained, joined). Stage outputs (transcript turns, entity lists, facts, grounding report) are nested documents we always read as a whole, so they go in **JSONB** (binary JSON: parsed once, indexable with GIN, queryable with operators like `->` and `@>`). A table per stage would add joins without any benefit for this app.
- **Append-only note versions = audit trail**: an edit never overwrites; it inserts version N+1 with the author. The LM's original is kept twice (version 1 and `generated_note`), so we can always show what the AI wrote vs what the clinician signed. Approval locks the note (409 on further edits).
- **Foreign key + ON DELETE CASCADE**: deleting a consultation deletes its versions; orphans are impossible.
- **timestamptz**: all times stored as absolute instants; the connection uses `timezone=utc`. We hit this bug during testing: naive `timestamp` columns showed times shifted by +05:30 (the Mac's local zone).
- **Index** on `status` (used by the start-up recovery query) and on the FK.

SQLAlchemy ORM concepts

- **Engine** = connection pool to PostgreSQL (driver: psycopg 3). `pool_pre_ping` tests a connection before use, so a restarted DB doesn't cause errors.
- **Session** = unit of work: collects changes and writes them in one transaction on `commit()`. If anything fails before commit, closing the session rolls back.

- **flush vs commit**: in `start_job` we `flush()` to get the new row's id (needed to name the upload file consult_<id>) without committing; if saving the file fails, the row is rolled back — no row without a file.
- **Mapped[...] / mapped_column** (SQLAlchemy 2.0 typed style): Python type hints define column types; `relationship()` gives `c.versions` as a list.
- `create_all` creates missing tables at start-up. In production you would use **Alembic migrations** so schema changes are versioned and reversible.

5. Streamlit — interactive clinical documentation

Streamlit's execution model

A Streamlit script is re-run **from top to bottom** on every interaction (button click, text edit). Values that must survive reruns live in `st.session_state` (selected consultation, last seen status). Widgets with a `key` keep their value across reruns.

- **Sidebar**: upload audio or paste a transcript ("Load example" fills one) → POST to the API.
- **Live progress**: `st.fragment(progress_panel, run_every=3)` re-runs only the progress block every 3 s (short polling) while a job is queued/running. When it sees the job finish, it triggers a full-page rerun to show the results.
- **SOAP note tab**: grounding check result, fact coverage, each grounding flag as a warning, an editable text area with the latest version, "Save version" (needs the clinician's name and a real change), "Approve" (locks it), and a Markdown download.
- **Transcript tab**: chat-style DOCTOR/PATIENT turns with timestamps. **Clinical entities**: BioBERT entities as a table. **SOAP facts**: the facts the LM was allowed to use. **Version history**: every saved version.
- **Human in the loop**: the AI drafts, the clinician decides. The UI keeps the AI's limits visible (flags, facts not used, model name) instead of presenting the note as final.

6. What each new file does

File	Role
app/database.py	Engine (DATABASE_URL), ORM models Consultation + NoteVersion, init_db(), get_db() dependency
app/pipeline.py	Stage list for audio (7) and transcript (3) inputs; async run_pipeline() with subprocesses, semaphore, progress updates, result import; mark_interrupted_jobs()
app/main.py	FastAPI app: lifespan, Pydantic schemas, 9 REST endpoints, validation, status codes
streamlit_app.py	UI; talks to the API only (API_URL env var)
requirements-app.txt	pinned versions: fastapi, uvicorn, python-multipart, SQLAlchemy, psycopg, streamlit, httpx
uploads/, logs/	git-ignored: uploaded patient data and per-job logs never get committed

7. How we tested it

- Validation: non-DOCTOR/PATIENT line → 422; README.md as audio → 415; unknown id → 404.

- Full runs: example transcript (3 stages) and sample_consultation_2.wav (7 stages) both reached 'completed'; the second waited as 'queued' while the first ran (semaphore). Results, grounding and version 1 stored in PostgreSQL.
- Review flow: save v2 → approve → further edit returns 409; version list shows LM v1 and clinician v2.
- Streamlit tested headless with `streamlit.testing.v1.AppTest` against the live API: no exceptions, progress bar during a run, metrics/tabs after completion, edit → save → approve through the widgets.
- Responsiveness: GET /consultations answered in 12 ms while the pipeline was running.

8. Limitations and how to make it production-grade

Limitation now	Production approach
No authentication; anyone who reaches the port can read notes; author is a typed name	OAuth2/OIDC login (hospital SSO), JWT on every request, role-based access, author taken from the token
Jobs live inside the web process (lost on restart, 1 machine)	Task queue: Celery / RQ / Arq with Redis or RabbitMQ; GPU worker machines; retries; API only enqueues
Polling every 3 s	WebSockets or Server-Sent Events push status changes
Files on local disk	Object storage (S3) with encryption and retention rules
create_all	Alembic migrations
Patient data in plain DB on a laptop	HIPAA/GDPR: encryption at rest and in transit (TLS), audit logs of every read, access controls, de-identification
Models reload per job	Long-lived model server (e.g. vLLM / Triton) for Qwen and Whisper
Manual tests	pytest + FastAPI TestClient with a test database; CI pipeline

9. Interview questions and answers

Q: Walk me through what happens when a clinician uploads a recording.

Streamlit POSTs the file to /consultations/audio. FastAPI checks the extension and size, inserts a row (flush to get the id), streams the file to uploads/consult_<id>.wav, commits, schedules run_pipeline as a background task and returns 202 with the id. The job waits on a semaphore, then runs the seven stage scripts as child processes, updating status/current_stage in PostgreSQL after each. When generate_soap_note.py finishes, the JSON outputs are copied into JSONB columns and version 1 of the note is inserted. Streamlit polls the detail endpoint every 3 s and shows the results when the status becomes completed.

Q: Why return 202 instead of waiting for the result?

The work takes minutes. Holding the request open risks client/proxy timeouts, ties up a connection and gives no progress information. 202 means 'accepted for processing'; the client gets an id to poll. This is the standard async job pattern for long-running operations.

Q: Is your app really asynchronous if the models run synchronously?

Yes, at the server level. The models run in child processes; the server awaits them with asyncio.create_subprocess_exec / communicate, which suspends only that coroutine. The event loop keeps serving requests — GET /consultations took 12 ms during a run. Inference itself is CPU/GPU-bound and is parallel to the server because it's a separate OS process.

Q: What would happen if you called the model directly inside an async endpoint?

The forward pass doesn't await anything, so it would block the event loop: every other request, including health checks and status polls, would hang until it finished. You'd have to use run_in_executor/to_thread, a process pool, or a separate worker.

Q: Why BackgroundTasks and not Celery?

For a single-machine demo, BackgroundTasks needs no extra infrastructure. Its limits: jobs die with the process, no retries, no distribution across machines. For production I'd use Celery/RQ/Arq with Redis — the API would only enqueue, GPU workers would consume, and job state would still be in PostgreSQL. The API contract (202 + polling) wouldn't change.

Q: Why a semaphore of 1?

Each job loads up to ~8 GB (Qwen3-4B bf16) plus Whisper/pyannote/BioBERT. Two at once could exhaust memory and crash both. The semaphore turns concurrent uploads into a FIFO-ish queue; the UI shows 'queued'. With more hardware you'd raise the limit or add workers.

Q: def vs async def in FastAPI?

async def runs on the event loop — only correct if all slow calls are awaited. def runs in a thread pool. Our CRUD endpoints use the synchronous SQLAlchemy session, so they're def. A blocking call inside async def is a common bug.

Q: Why PostgreSQL and not SQLite or MongoDB?

PostgreSQL gives concurrent writers (the background job and API requests write at the same time), real transactions and foreign keys for the audit trail, timezone-aware timestamps, and JSONB — so I get document-style storage for nested NLP outputs without giving up relational guarantees. SQLite locks the whole DB on writes; MongoDB would lose easy relational integrity between notes and versions.

Q: JSONB vs JSON in PostgreSQL?

JSON stores the text as-is (keeps whitespace/key order, re-parsed on every read). JSONB stores a parsed binary form: slightly slower to insert, much faster to query, supports GIN indexes and containment operators. JSONB is the usual choice.

Q: How do you keep an audit trail of note edits?

Edits are append-only rows in note_versions with version number, author, comment and timestamp. Nothing is updated in place; version 1 is always the model's output. Approval records who and when, and the API refuses further edits with 409. In production the author would come from the authenticated user, not a text field.

Q: Explain flush() vs commit().

flush sends pending SQL to the database inside the open transaction (so we get the auto-generated id) but other sessions can't see it yet; commit ends the transaction and makes it durable/visible. We flush, save the file named with the id, then commit — if the file save fails, the row is rolled back.

Q: How do you handle a server crash in the middle of a job?

The job is in-process, so it is lost. On start-up the lifespan hook marks queued/running rows as failed with a clear message so the UI never shows a job 'running' forever. A durable queue with acknowledgements and retries would fix it properly.

Q: How does Streamlit show live progress without reloading the page?

st.fragment with run_every=3 re-executes only the progress function every 3 seconds while the job is running; it calls the API and updates the progress bar. When it sees the status change to completed, it triggers a full-app rerun. It's short polling; WebSockets or SSE would push updates instead.

Q: Why doesn't Streamlit talk to the database directly?

Separation of concerns: validation, state rules (no edits after approval) and job orchestration live in one place, the API. Any client gets the same rules, the DB credentials stay on the server, and the UI can be replaced.

Q: What security issues does this app have?

No authentication or authorization, no TLS, author is self-declared, uploads are stored unencrypted, and notes contain PHI. For real use: SSO/OAuth2 with JWT, role-based access, TLS, encryption at rest, audit logging of reads, data retention policies, and HIPAA/GDPR compliance review. File uploads are limited by extension and size, and saved under server-generated names (consult_<id>) — never the user's file name — to avoid path traversal.

Q: How would you scale this to a hospital?

Stateless API replicas behind a load balancer; job queue + GPU worker pool (or a model server like vLLM for the LM and batched Whisper); object storage for audio; managed PostgreSQL with backups/replicas; WebSocket/SSE notifications; observability (structured logs, metrics per stage, tracing); Alembic migrations and CI/CD.

Q: How would you test the API?

pytest with FastAPI's TestClient and a separate test database (override get_db via app.dependency_overrides); mock run_pipeline so tests don't load models; test validation codes (415/422/404/409), the versioning rules and approval lock. A small end-to-end test with the synthetic sample clip in CI (nightly, since it's slow).